ClusterIP 长连接：从建连到持续收发

Flannel、Calico、Cilium 的 Service 选择、跨节点数据路径与连接状态

核心结论：一条 TCP 连接建立时选定后端，后续数据通常持续发往同一个 Pod。网络模型决定数据包如何送达；Service 实现决定在哪里选后端、如何保持映射。

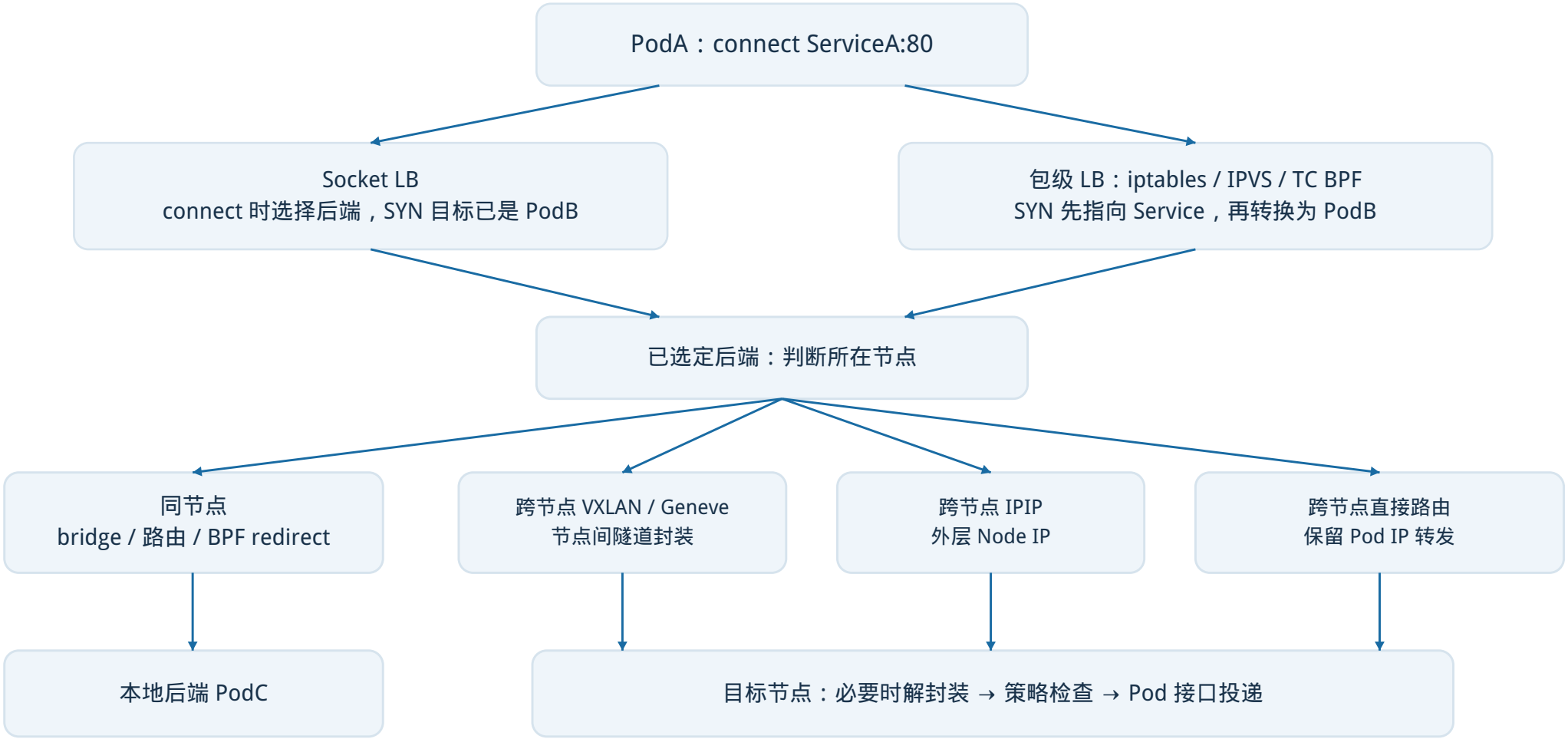
对象	地址	位置
客户端 PodA	10.244.1.10:40000	Node1：192.168.1.11
ServiceA	10.96.0.100:80	虚拟服务地址，targetPort 为 8080
后端 PodB	10.244.2.20:8080	Node2：192.168.1.12
后端 PodC	10.244.1.30:8080	Node1，与 PodA 同节点

阅读约定

Linux / TCP / 普通 Pod 网络；无 Sidecar 或 L7 代理，无额外加密；集群内 Pod 通信不做额外 SNAT。图中省略重传、ACK 合并和网卡 offload，展示关键逻辑阶段。A、B、S 分别代表 PodA、PodB、ServiceA。

01 / 整体模型：先选后端，再确定转发路径

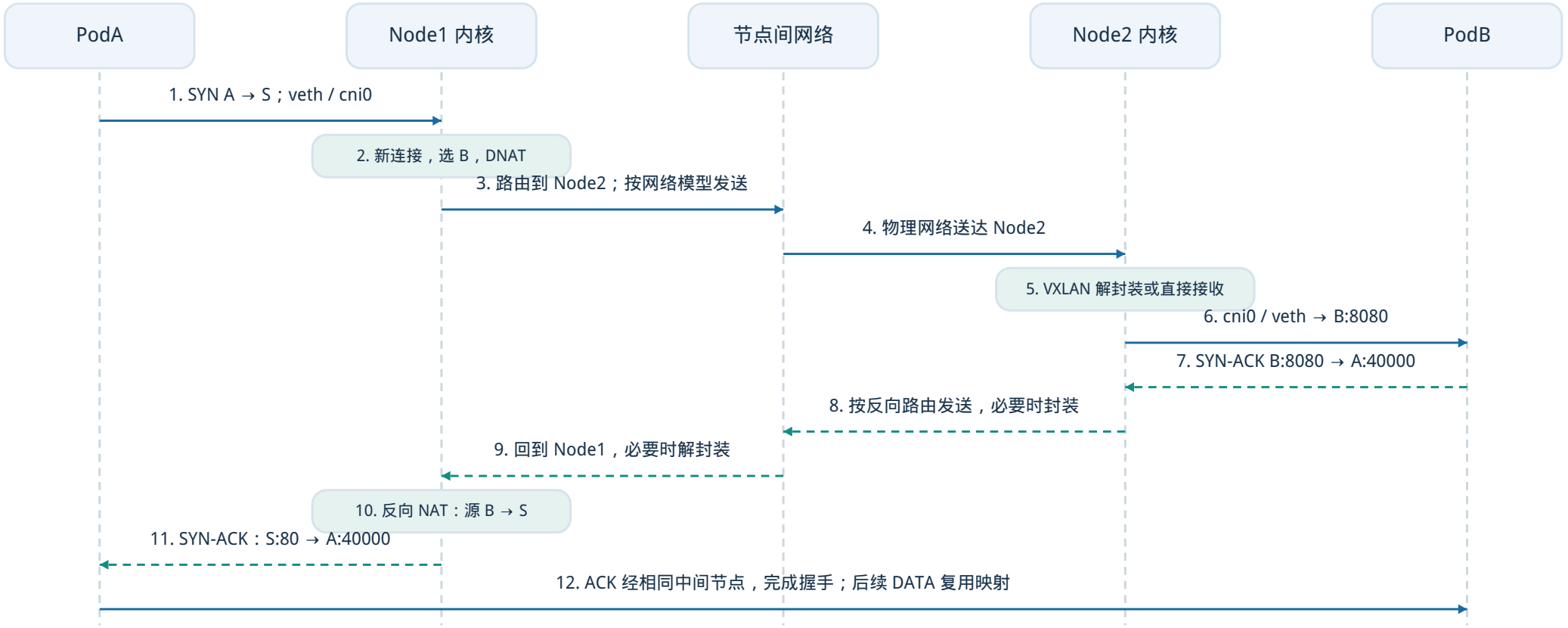
ClusterIP 是逻辑入口；kube-proxy 和网络 agent 下发规则，普通业务包由内核或 BPF 数据面处理。[1][5]



两个独立维度：使用 eBPF 不代表没有隧道；使用 BGP 不代表没有封装。

02 / Flannel：VXLAN 与 host-gw

假设 kube-proxy 使用 iptables。首次 SYN 选 PodB，Node1 conntrack/NAT 保存连接映射。[1][2]



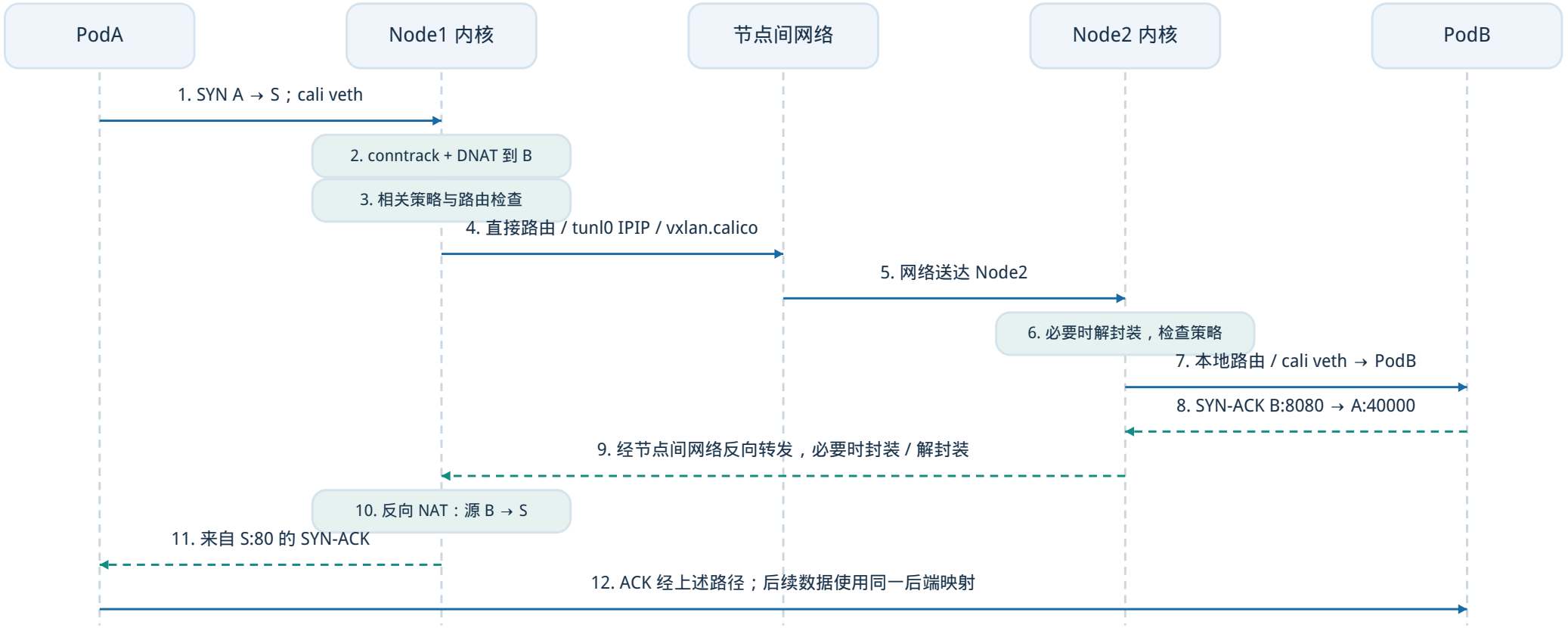
VXLAN：flannel.1 封装；外层 Node1 → Node2，内层 A → B。Linux 默认 UDP 8472。

host-gw：不封装，Pod IP 目标仍是 B；以 Node2 为路由下一跳，要求节点直接二层可达。

DirectRouting：同子网可直接路由，跨子网使用 VXLAN。隧道外层 Node IP 不等于对 Pod 做 SNAT。

03 / Calico 标准数据面：三种跨节点模型

假设 kube-proxy iptables；典型 cali* veth + 三层路由，通常不使用 cni0 网桥。[1][3][4]



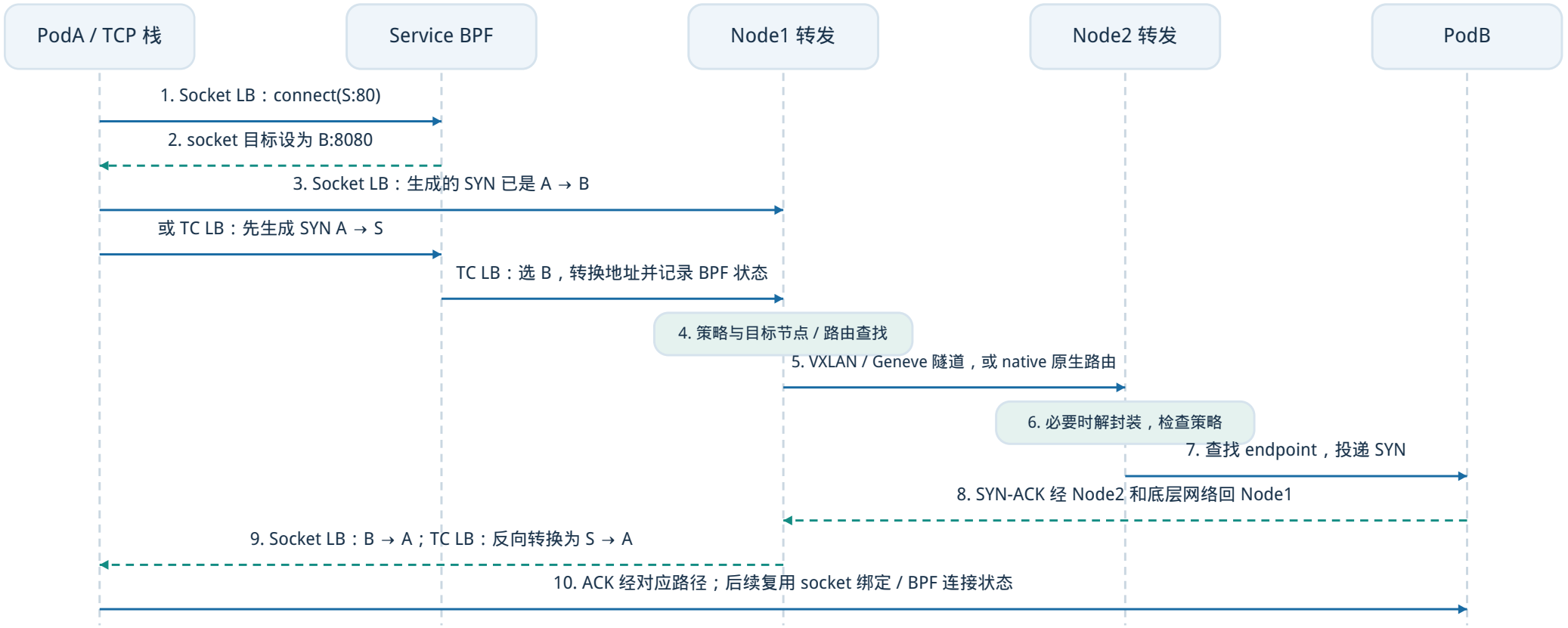
直接路由：传输原始 Pod IP 包；常用 BGP 发布 Pod 网段。IPIP：外层 Node IP，内层 Pod IP。

VXLAN：vxlan.calico 封装；VXLAN-only 网络可不使用 BGP。CrossSubnet 仅跨节点子网时封装。

BGP 是路由控制协议，业务包不会经过 BGP 进程。网络策略在相关接口 / 转发点执行，图中合并表示。

04 / Cilium：Socket LB、TC LB 与转发模型

假设启用 kube-proxy replacement；Service 处理位置与是否隧道封装是两个独立选择。[5][6]



Tunnel：基础网络只需能到达节点地址；Native routing：底层及节点路由需能到达 Pod IP / 网段。

Socket LB 免除这条连接的包级 Service NAT，但策略等功能仍可能使用 BPF 连接跟踪。

Calico eBPF 也支持 TCP connect-time LB：在 connect() 阶段选后端，不能把所有 Calico 都视为 iptables 模型。[7]

05 / 地址变化：每个观察点看到什么？

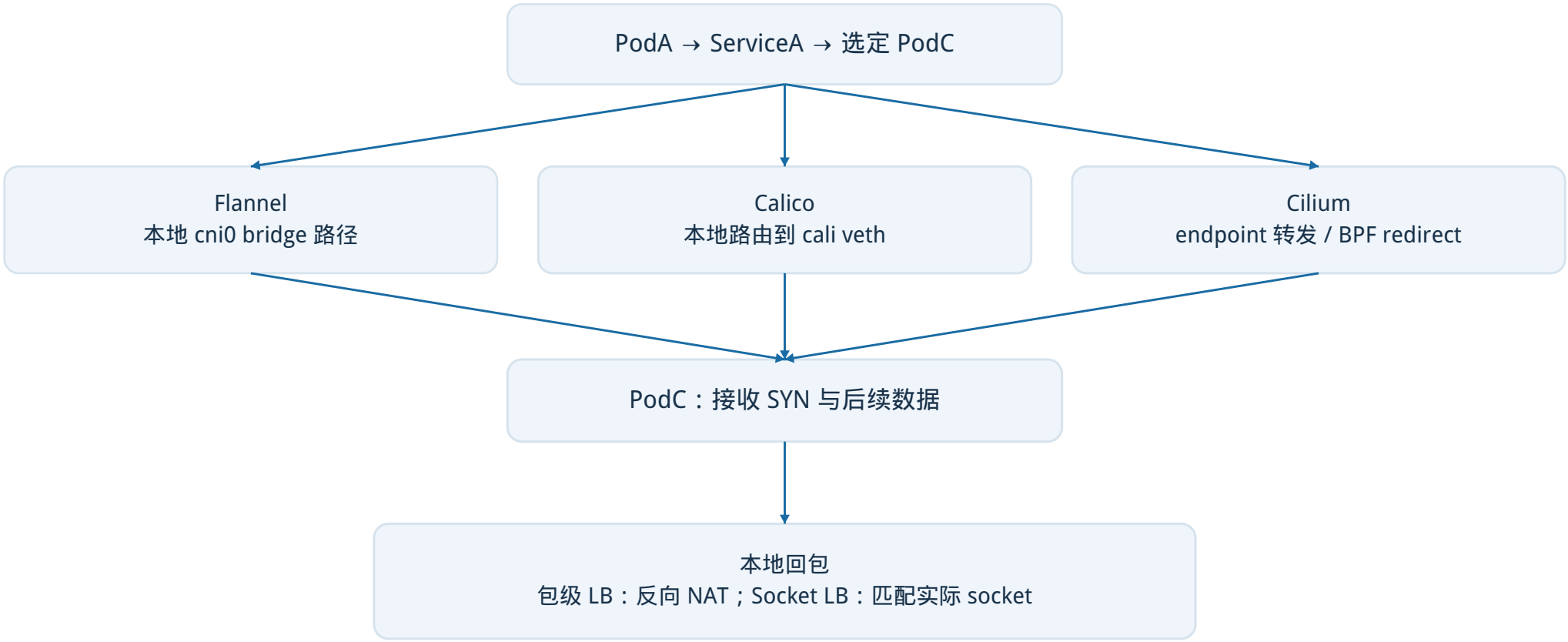
跨节点 VXLAN 示例。A = PodA，B = PodB，S = ServiceA；无额外 SNAT。

观察位置	iptables / TC 包级 LB	Socket LB
应用 connect()	请求 S:80	请求 S:80
PodA TCP 栈生成 SYN	A:40000 → S:80	A:40000 → B:8080
Node1 Service 处理后	A:40000 → B:8080	A:40000 → B:8080
Node1 物理网卡发出	外层 Node1 → Node2；内层 A → B	外层 Node1 → Node2；内层 A → B
Node2 解封装 / PodB 收到	A:40000 → B:8080	A:40000 → B:8080
PodB 回复	B:8080 → A:40000	B:8080 → A:40000
Node2 物理网卡发出	外层 Node2 → Node1；内层 B → A	外层 Node2 → Node1；内层 B → A
Node1 解封装后	B:8080 → A:40000	B:8080 → A:40000
投递 PodA TCP 栈	反向转换：S:80 → A:40000	保持 B:8080 → A:40000

直接路由模式去掉隧道外层；包级 LB 仍然执行 Service 地址转换。Socket LB 下抓 Pod 网卡，可能已经只看到后端 Pod IP。

06 / 同节点：选中本地后端 PodC

同节点不需要跨节点隧道，不需要经过物理交换机；后端选择步骤仍然存在。

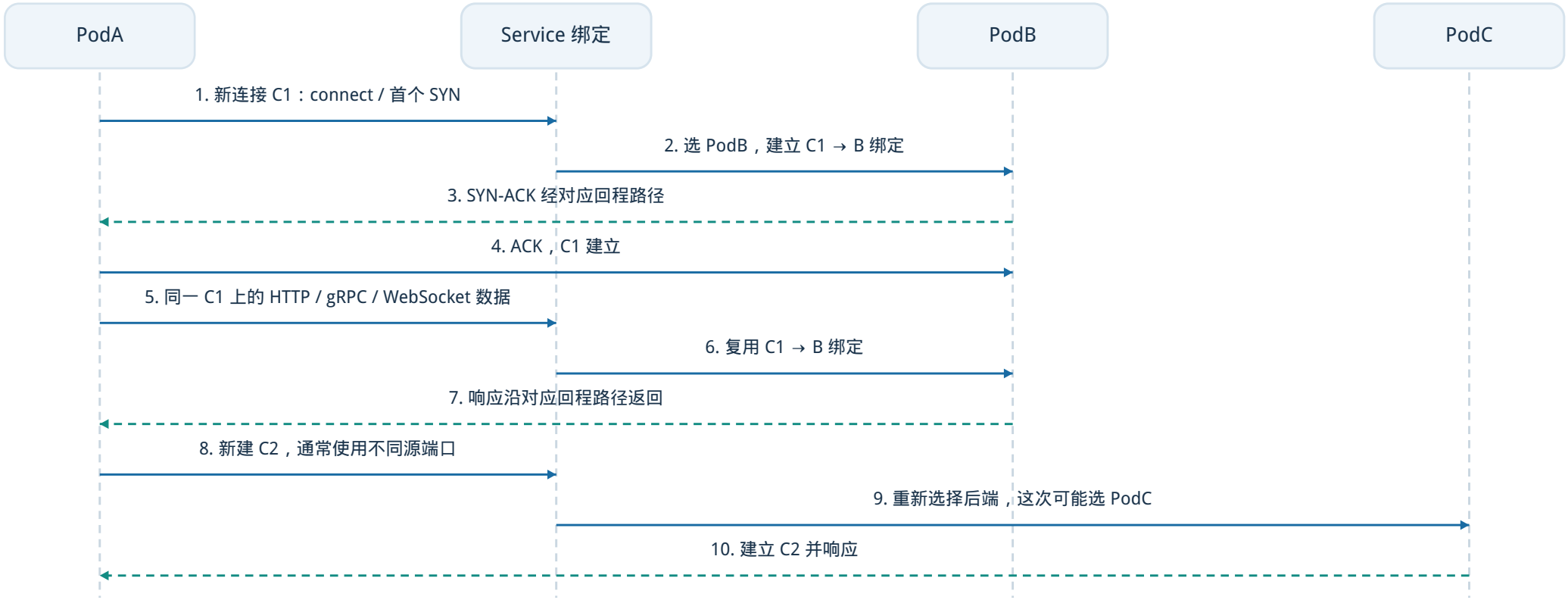


同节点后端不代表默认一定优先选它。后端选择还受 Service 流量策略及实现影响。

本例 PodC 与 PodA 不同；若客户端自己就是选中的 Service 后端，还需考虑 hairpin 及相关 SNAT 行为，不能直接套用本页的无 SNAT 假设。

07 / 长连接保持：绑定后端，不逐请求重新选择

普通 ClusterIP 的 TCP 负载均衡以连接为单位；HTTP/2 stream 不会自动变成独立的后端选择。



扩容增加 PodC，通常不会使健康的既有连接 C1 自动迁移。后续包仍要转发；包级 NAT 也仍要应用转换，只是不重新选择后端。

sessionAffinity: ClientIP 影响同一客户端的多条连接如何选后端；维持单条 TCP 连接的后端一致性不依赖它。

08 / 连接状态、心跳与故障边界

连接"保持"需要区分 TCP 状态、Service 绑定和网络可达性三个层次。

层次	保存内容	作用
两端 TCP socket	序列号、窗口、重传及连接状态	可靠、有序的字节流
Service 连接绑定	conntrack NAT / IPVS 连接项 / BPF 状态，或 socket 的实际后端地址	使同一连接持续到达同一后端
Pod 网络	路由、邻居、隧道映射、网络策略	把每一个数据包送达目标

iptables 连接映射示例

原始：10.244.1.10:40000 → 10.96.0.100:80
实际：10.244.1.10:40000 → 10.244.2.20:8080
回复：10.244.2.20:8080 → 10.244.1.10:40000
客户端收到：10.96.0.100:80 → 10.244.1.10:40000

- Service / CNI 不会替应用自动发送心跳。空闲连接受 TCP keepalive、应用心跳和路径状态超时影响。
- 后端故障后，既有 TCP 连接不能直接迁移到另一 Pod；客户端要检测失败并重新连接。
- 扩容主要影响新连接。长连接服务仅增加副本，不一定马上获得均匀负载。
- 如果经过 Istio / Envoy，需要加入应用到代理、代理到后端的连接层，不能直接套用无代理路径。

09 / 模型对照与参考资料

按常见组合整理；实际数据路径以集群版本、数据平面和运行配置为准。

网络方案	Service 处理	跨节点传输	既有连接绑定
Flannel + kube-proxy	iptables / IPVS 等	VXLAN / host-gw	conntrack NAT / IPVS 连接项
Calico 标准数据面	kube-proxy	直接路由 / IPIP / VXLAN	同对应 kube-proxy 实现
Calico eBPF	connect-time LB / BPF	由运行配置决定	socket 后端 / BPF 状态
Cilium kube-proxy replacement	Socket LB / TC LB	VXLAN / Geneve / native	socket 后端 / BPF 状态

官方参考资料（对应正文编号）

[1] Kubernetes - Virtual IPs and Service Proxies

[2] Flannel - Backends

[3] Calico - Overlay networking

[4] Calico - Data path

[5] Cilium - Kubernetes Without kube-proxy

[6] Cilium - Routing

[7] Calico - About eBPF

文档整理日期：2026-10-13。图为逻辑数据路径，不代表所有版本的完整内核 hook 顺序。